

Image Captioning Project

Documentation & Technical README

Project Overview

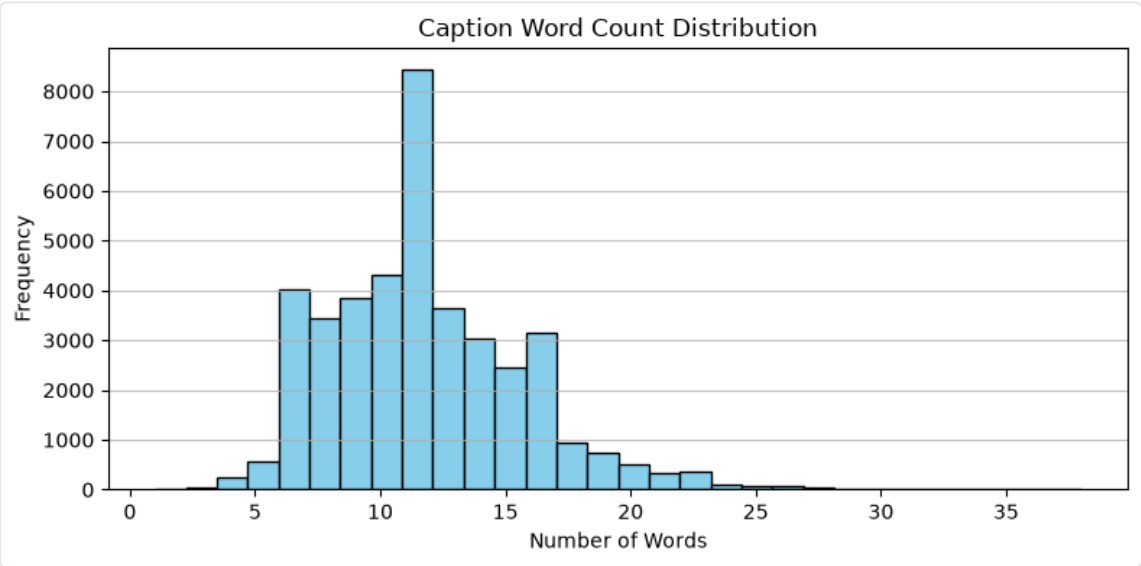
A model that takes an image as an input and outputs a caption that describes that image where a ResNet-50 CNN encoder extracts the features of the input image and passes it to an LSTM decoder with additive attention.

The Backend of the project is handled by FastAPI which interacts with a Gradio web UI.

Dataset

Flickr8k Dataset Summary

- **Total Captions:** 40,455
- **Unique Images:** 8,091
- **Average Word Count:** 11.78 words/caption
- **Max Word Count:** 38 words
- **Min Word Count:** 1 word
- **Captions per Image:** Exactly 5 text descriptions



Splitting was done by a `GroupShuffleSplit` grouped by image filename so that all 5 captions stay together avoiding leakage.

Split	% of images	Images	Captions
Train	70%	5,663	28,315
Validation	15%	1,214	6,070

Split	% of images	Images	Captions
Test	15%	1,214	6,070

Architecture

Encoder

- Pretrained ResNet-50 (ImageNet weights), with the final average-pool and fully-connected layers removed.
- Output feature map (Batch, 2048, 7, 7) is reshaped into a spatial sequence (Batch, 49, 2048).

Attention

Additive attention compares the 49 encoder output vectors to the hidden state of the current step of the decoder and gets the weighted sum context vectors.

Decoder

A word embedding layer passes the word vector to a single LSTM cell that processes the hidden state that would be used with the weighted sum context vector to predict the suitable word.

Note: The LSTM hidden state and cell state are initialized from the mean of the encoder features.

Hyperparameters (deployed model)

Parameter	Value
Embedding size	256
LSTM hidden size	512
Attention dimension	256
Encoder feature dimension	2048
Decoder dropout	0.3
Vocabulary	Built from training captions (frequency threshold ≥ 2)

Preprocessing

Text

- **Vocabulary class:** Lowercases text and tokenizes with a simple regex word tokenizer.
- **Special tokens:** <PAD> , <SOS> , <EOS> , <UNK> .
- Vocabulary is built only from the training split where a word must appear at least twice to get its own token.

Images

Resize each image to 224×224 and convert it into a tensor + normalizing the tensor.

Training Process

Train Encoder Decoder Attention with:

- **Loss:** Cross-entropy, ignoring `<PAD>` tokens.
- **Optimizer:** Adam, learning rate 0.0003.
- **Epochs:** 10
- **Teacher forcing:** The decoder is fed the ground-truth caption (shifted by one token) at every training step; the target is the caption shifted one further token to the right.

Training loss (attention model, per epoch)

Epoch	1	2	3	4	5	6	7	8	9	10
Avg. loss	4.13	3.27	2.93	2.69	2.50	2.34	2.19	2.07	1.96	1.85

Evaluation Metrics & Results

Metric: Corpus-level BLEU-1 to BLEU-4, computed by generating one greedy caption per unique image and comparing it against all 5 human reference captions for that image.

Model	BLEU-1	BLEU-2	BLEU-3	BLEU-4
Approach 1: CNN RNN (no attention), 5 epochs	48.52	18.53	2.68	0.45
Approach 2: CNN + Bahdanau Attention (deployed), 10 epochs	59.75	41.82	28.48	18.40

How to Install and Run the Project

Prerequisites

- Docker
- The trained model weights and vocabulary which are uploaded to my HuggingFace repo: <https://huggingface.co/faressaad297/Image-Caption>

Before building the image or running the docker container make sure these exist at the project root:

- `data/Images/`
- `data/captions.txt`
- `weights/train_attention_enc_dec1.pth`
- `splits_vocab/vocab.pt`

```
# Build the image
docker build -t image-captioning .

# Run the container (exposes the API on 8000 and the UI on 7860)
docker run --rm -p 8000:8000 -p 7860:7860 image-captioning
```

Start the API + UI by:

```
./start.sh
```

which starts:

- FastAPI backend on `http://localhost:8000` (`uvicorn app.main:app`)
- Gradio UI on `http://localhost:7860` (`app/interface.py`), which internally calls the FastAPI `/caption` endpoint.

You can also run each independently for development:

```
# API only, with auto-reload
uvicorn app.main:app --reload

# UI only (requires the API to already be running)
python app/interface.py
```

How to Use Application / API

Web UI (Gradio)

- Open `http://localhost:7860`
- Upload an image
- Click "Generate Caption"

REST API (FastAPI)

Interactive Swagger docs are auto-generated at `http://localhost:8000/docs` .

Method	Endpoint	Description
GET	<code>/</code>	Welcome message
GET	<code>/health</code>	Simple health check → <code>{"status": "ok"}</code>
POST	<code>/caption</code>	Upload an image, get back a generated caption

Inputs & Output Example

Image ID: `1067180831_a59dc64344.jpg`

Image ID: 1067180831_a59dc64344.jpg



=== Results for Sample #46 (1067180831_a59dc64344.jpg) ===

Ground Truth Caption(s):

1. A black and white dog jumps after a yellow toy.

Generated Caption:

a black and white dog is running on the grass